Old early papers that doesn't mean too much.

# Robot Navigation in Constrained Pedestrian Environments using Reinforcement Learning

Claudia Pérez-D'Arpino, Can Liu, Patrick Goebel, Roberto Martín-Martín, Silvio Savarese

arXiv:2010.08600v2 [cs.RO] 16 Nov 2020

***Abstract*— Navigating fluently around pedestrians is a necessary capability for mobile robots deployed in human environments, such as buildings and homes. While research on social navigation has focused mainly on the scalability with the number of pedestrians in *open spaces*, typical indoor environments present the additional challenge of *constrained spaces* such as corridors and doorways that limit maneuverability and influence patterns of pedestrian interaction. We present an approach based on reinforcement learning (RL) to learn policies capable of dynamic adaptation to the presence of moving pedestrians while navigating between desired locations in constrained environments. The policy network receives guidance from a motion planner that provides waypoints to follow a globally planned trajectory, whereas RL handles the local interactions. We explore a compositional principle for multi-layout training and find that policies trained in a small set of geometrically simple layouts successfully generalize to more complex unseen layouts that exhibit composition of the structural elements available during training. Going beyond walls-world like domains, we show transfer of the learned policy to unseen 3D reconstructions of two real environments. These results support the applicability of the compositional principle to navigation in real-world buildings and indicate promising usage of multi-agent simulation within reconstructed environments for tasks that involve interaction.**
**https://ai.stanford.edu/∼cdarpino/socialnavconstrained/**

## I. Introduction

As mobile robots become more capable of executing a variety of tasks, they are becoming increasingly likely to be deployed in environments co-located with people. This step demands seamless and safe integration into spaces with pedestrians such as buildings, homes and offices. In the path towards socially compliant mobile robots, research on human-robot co-navigation has made progress on planning and learning approaches for robots that can handle local interactions with moving pedestrians among crowds in open spaces. Navigation in indoor environments presents additional challenges due to space constraints that limit maneuverability around the pedestrians and create layout-dependent patterns of pedestrian interaction. These spaces, such as corridors, doors and intersections pose hard problems for any existing navigation solution. For these types of spaces, there is a need for new interactive navigation models for tight spaces with constraints imposed by obstacles, walls, doors and other common structural elements, based only on on-board sensor information. Fig. 1 illustrates some of the basic layouts and the common interactions that emerge in these geometries from typical indoor environments.

Authors are with the Department of Computer Science at Stanford University.

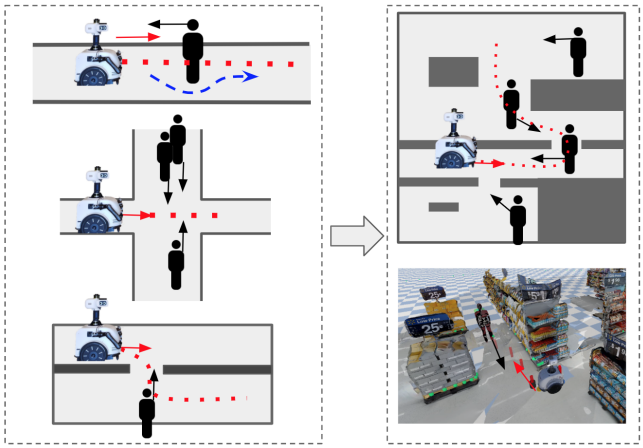


Fig. 1: Illustration of the problem of robot navigation around pedestrians in constrained indoor environments. The robot follows a global path (red dotted line) while maneuvering to circulate seamlessly around pedestrians (blue dashed line example). The robot's policy is trained on a small set of simple walls-world environments representative of common indoor layouts and the emerging interactions, such as in corridors and intersections (left). We show generalization to two types of unseen environments (right): more complex compositional walls-worlds (top right) and 3D reconstructions of real-world sites (bottom right).

In this paper we present a novel robot learning approach and training strategy focusing in the ability to navigate around pedestrians in typical indoor environments in which spatial constraints are sufficiently important to influence the feasibility of the robot's behavior. The proposed architecture takes advantage of the complementary strengths of motion planning and reinforcement learning (RL), in which the RL component learns to handle the local interactions with pedestrians as it pursues the globally planned trajectory, and adapts to the current conditions of the environment as observed with on-board sensors, even in constrained indoor spaces such as corridors and doors. We observe that typical indoor environments exhibit common geometric elements and diverse composition of these elements. With the aim of generalization to novel realistic spaces, we explore a compositional principle for multi-layout training, in which a policy is trained in a small number of simple, canonical, layouts, such as a corridor, or a single door exit (Fig. 1 left). We analyze the capability of this approach to generalize to more complex layouts with multiple composed and combined navigation challenges not seen during training (Fig. 1 top right), and find experimentally that policies trained on a set simple layouts generalize better when those represent the elements of the geometries and interactions that conform to

the target layout. This insight results particularly relevant for the built environment, which is typically composed by combinations of basic layouts templates, such as hallways, rooms, door exits, and crosswalks. We then demonstrate transfer of the policies learned on walls-worlds (Fig. 1 left) to unseen 3D reconstructions of two real-world environments (Fig. 1 bottom right), such as a supermarket and an apartment.

We instantiate our navigation approach and train it to control a simulated non-holonomic wheeled robot in multiple environments on the interactive Gibson simulator [1][2], which leverages the Pybullet physics engine [3] for realistic physics simulation and supports virtual sensing (lidar) for our multi-agent setup. The environment includes simulated pedestrians driven by the Optimal Reciprocal Collision Avoidance (ORCA) model [4] based on a social forces model [5]. Fig. 2 illustrates the two types of simulation environments used in this paper: the simple walls-worlds layouts implemented using simple shapes, denominated by the prefix WALLS- (Fig. 2 left), and the real-scale 3D reconstructed environments implemented using textured meshes from the Gibson dataset [1] (Fig. 2 right), denominated by the prefix MESH-.

In summary, this paper presents the following contributions: (1) a **learning approach and a simulation environment** for the problem of goal-oriented robot navigation around moving pedestrians, that unifies ideas from reinforcement learning, planning and multi-agent simulation to enable human-robot co-navigation in constrained environments representative of real human indoor spaces; (2) **a compositional multi-layout training regime** using canonical walls-worlds layouts that generalizes to more complex environments; and (3) a **demonstration of transfer** of the learned policy on simplified walls-world **to unseen 3D reconstructions of two real-world environments** represented by scanned 3D textured meshes. These results support the compositional principle applicability to real-world environments and indicate promising frontiers for the emerging topic of agent simulation within reconstructed environments for tasks that involve human-robot interaction. Accompanying the paper, we present a video [6] with demonstrations of the robot behaviors obtained in the experiments.

## II. Related Work

***Navigating safely among humans*** is a prerequisite for the deployment of mobile robots. This has motivated researchers in autonomous navigation and human-robot interaction to study different aspects of the navigation problem, such as navigation within crowds [7], communicating implicit and explicit navigation information with haptic devices [8], co-navigation [9] using anticipatory signals [10], navigation cues in human gaze [11], and the effects of distinct robot navigation strategies on pedestrians [12]. Despite this long tradition, truly social mobile robots that are safe, efficient and fluent when deployed in human environments are still a research challenge.

Early solutions **for navigation in human environments** combine a motion planner [13] with a controller that executes the plan [14]. These approaches suffer from the "freezing robot problem" [15], [16], in which the planner finds all solutions to be below certain threshold of safety or feasibility, causing the robot to stop and continuously replan while the pedestrians move. This problem is more severe in domains with complex geometry and large number of pedestrians. To overcome these limitations, researchers have resort to machine learning approaches [17]. Multiple approaches have proposed to use imitation learning to match human navigation patterns [18][19][20][21]. The limitation of imitation learning is that the agent does not generalize well beyond what has been seen in the demonstrations, resulting on potentially unsafe actions when the robot reaches states out of distribution. Reinforcement learning (RL) [22] with a self-exploratory loop in simulation is a technique that naturally provides further coverage of the state space.

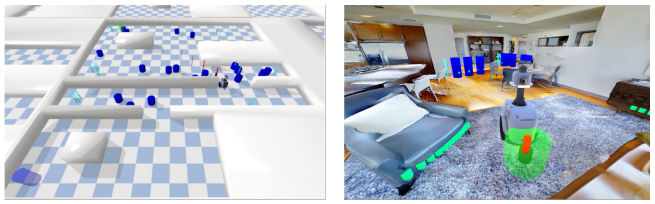



Fig. 2: Implemented multi-agent simulation environments. Layouts with the prefix **WALLS-** are simple walls-worlds (left). The prefix **MESH-** is used for 3D reconstructions from real sites (right).

Previous works pioneered the use of ***deep reinforcement learning*** *in navigation around pedestrians* and developed multi-agent collision avoidance systems focused on optimizing path efficiency (CADRL)[23] and using the reward function to induce social norms such as passing on the right (SA-CADRL)[24]. Deep RL has also been successfully used for crowd navigation with recursive architectures that model interactions among pedestrians in the crowd to improve the model's performance as the number of pedestrians becomes large in open spaces (SARL)[7]. These approaches use pedestrians positions as input to the network (either ground-truth in simulation or detection/tracking from vision and lidar), but do not explicitly consider the presence of geometric constraints in the layout and are rather tested on spacious environments with no sensor input that reflects environmental constraints for the local interactions. This paper proposes a lidar-based framework with an architecture and training regime that enables the policy to acquire the behaviors needed to perform in constrained indoor environments.

Previous solutions have explored how to *combine* ***motion planning and learning*** for navigation around dynamic obstacles. The idea is to leverage the motion planner to provide guidance to a learned controller, which adapts the plan to the current surrounding conditions. This multi-level strategy to combine the long-term planning (global path in the map) and short-term decision making also resembles the principles of how humans perform spatial navigation tasks [25]. [21] and [26] showed this structure in imitation learning outperforms a single monolithic end-to-end architecture with no planning guidance, and [27] combined it with RL and active learning

in order to learn human preferences for speed. We build on these ideas and propose and architecture and training regime to use RL instead of imitation learning to avoid being limited to the regions of the state space covered in the expert demonstrations.

Finally, our learning procedure *trains on simple instances* of a task with the goal of generalizing to task instances with the true complexity. Similar idea is behind methods on curriculum learning [28] that create a series of increasingly complex task instances to facilitate learning [29][30][31]. Differently, we assume that the challenges of our original navigation task cannot be learned progressively; they are the result of combining a finite set of complex patterns of layout-motion of dynamic agents (pedestrians). Therefore, we propose to learn to overcome these complex patterns separately in independent but intertwined training episodes, which will enables us to transfer eventually to a navigation task with the full combinatorial complexity.

## III. Learning to Navigate in Pedestrian Environments

We propose to address the navigation problem in pedestrian environments with a combination of a sampling-base motion planner [13] and a reactive low level sensorimotor policy learned with RL and implemented as a deep neural network. We assume that our solution can make use of a 2D map approximating the layout of static elements in the environment (e.g. furniture and walls) and that it can localize on it. Each episode the robot will start at a different location and is queried to navigate to a new final goal defined as a location in the map. We query the motion planner once at the beginning of the episode for a shortest path to the goal location. This path information is then used to define the low-level sensorimotor task that we describe next, followed by a description of the simulation environments we create to train our low-level policy.

### A. Reactive Navigation to Follow a Path

We model the problem of navigating safely among humans following a given path as a Partially Observable Markov Decision Process (POMDP) defined by the tuple $\mathscr{M} = (\mathscr{S}, \mathscr{A}, \mathscr{O}, \mathscr{T}, \mathscr{R}, \gamma)$. Here, $\mathscr{S}$ is the state space; $\mathscr{A}$ is the action space; $\mathscr{O}$ is the observation space; $\mathscr{T}(s'|s,a), s \in \mathscr{S}, a \in \mathscr{A}$, is the state transition model defining a probability over the next state $s'$ after taking action $a$ in state $s$; $\mathscr{R}(s) \in \mathbb{R}$ is the reward at state $s$; $\gamma \in [0,1)$ is the discount factor. We assume that the state is not directly observable and learn a policy $\pi(a|o)$ conditioned on observations $o \in \mathscr{O}$ instead of latent states $s \in \mathscr{S}$. The agent following the policy $\pi$ obtains an observation $o_t$ at time $t$ and performs an action $a_t$, receiving from the environment an immediate reward $r_t$ and a new observation $o_{t+1}$. Assuming the policy is parameterized by $\theta$, a policy gradient algorithm optimizes $\theta$ to maximize that the expected future return:

$$\theta^* = \arg\max_\theta J(\theta) = \arg\max_\theta E_\pi \left[ \sum_t \gamma^t r_t \right] \qquad (1)$$

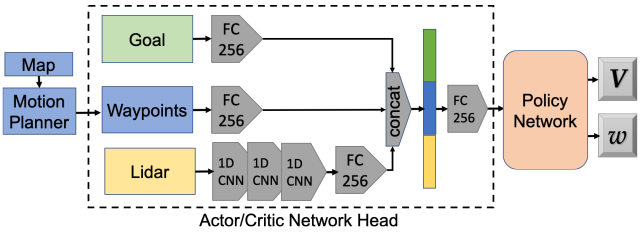



Fig. 3: Network Architecture.

More concretely, the observation space, $\mathscr{O}$, includes the elements $o = \{goal, lidar, waypoints\}$, where *goal* is the episodic navigation goal, represented by the 2D coordinates in robot's reference frame, *lidar* contains the 128 range measurements from a 1D LiDAR sensor in robot sensor reference frame, and *waypoints* contains a $n = 6$ waypoints computed by a global planner with access to a map of the environment. The action space, $\mathscr{A}$, defines actions $a = \{v_x, v_y, \omega\}$, where $(v_x, v_y)$ is the commanded linear velocity, and $\omega$ is the commanded angular velocity for the mobile robot.

The reward function is composed of multiple terms that encourage the policy to reach the final goal, minimizing time, following the given global path but deviating from it to avoid the penalized collisions:

$$R = R_{goal} + R_{timestep} + R_{collision} + R_{potential} + R_{waypoint} \qquad (2)$$

where $R_{goal} = +1$ is a sparse reward assigned upon reaching the goal within a distance $d_g = 0.5m$, $R_{timestep} = -0.001$ is a small negative reward with value per time step to encourage minimizing the task time, $R_{collision} = -1$ is assigned upon collision and terminates the episode, $R_{potential}$ is a dense reward assigned per time step as a function of the distance to the next desired waypoint.

We use the Soft Actor-Critic (SAC) algorithm [32], [33] to learn both a value function (critic) and a policy (actor) approximated by deep neural networks. SAC optimizes for a combined objective of the expected sum of rewards and the entropy. The policy network maps from observation space to action space, providing velocity references to a low level velocity controller. The model architecture we propose for the policy network is depicted in Fig.3. The critic network presents a similar architecture with a different implementation after the shared featurizing head. Through extensive ablation studies, we tested a number of other variants for the observation space by including other inputs to the policy network, such as observation stacking with consecutive lidar frames, robot odometry, pedestrian detection (coordinates of pedestrians) and time to collision. These variants did not offer significant performance advantages in our domains and we opted for the simplest model that solves the task. However, these other inputs are expected to become relevant when considering explicit aspects of social interactions, such as pedestrian prediction, semantics, human-like social patterns, preferences and safety guaranties. We believe this is an interesting avenue for future research that can build on the presented model and leverage our multi-agent simulation environment.

### B. Simulated Environments

We created a multi-agent simulation environment for the general problem of navigation around pedestrians, in which a simulated robot agent can collect experience and train according to the described POMDP. We use the Interactive Gibson Simulator (iGibson) [2] that runs on top of the pyBullet physics engine [3]. Figure 4 shows a top-down view of the environments used for training and testing. These layouts were designed to capture the essential geometric properties of many indoor environments such as corridors, crossing hallways and office spaces with doorways. Many indoor spaces can be viewed as a composition of these simpler components.

Pedestrian behaviors are simulated using ORCA [4]. This model is often used to simulate pedestrian motion for crowd simulation and robot social navigation research [7]. ORCA uses a joint optimization and a centralized controller that guarantees that pedestrians will not collide with each other or any other objects added to a list of known obstacles, an useful property for driving the simulated pedestrians and provide a training environment for the robot.

Based on a standard RL episodic training loop, a collision between the robot and any object results in termination of the episode and environment reset, which generates new random samples for the start and goal location of the robot and all pedestrians. Success is recorded if the robot gets to within 0.5 meters of the goal. A timeout is recorded if it takes the robot longer than 125 seconds to get to the goal. The personal space radius for each pedestrians was set to 10*cm*.

## IV. Experimental Evaluation and Results

We conduct a series of experiments in simulation using the environments in Fig.4 [1] [2] . We design the experiments to study the following questions: **(Sec. IV-A.1)** Does the proposed approach enable a robot agent to learn to navigate around pedestrians in constrained environments?; and does multi-layout training on a small set of canonical environments (WALLS-ABCDF) generalize to navigation on more complex layouts (WALLS-GHI) that exhibit composition of the basic geometric elements? **(Sec. IV-A.2)** How does the proposed method compare with a planning-only approach; and with **(Sec. IV-A.3)** previous work on RL for a feasible domain? **(Sec. IV-A.4)** Can a policy trained in the simplified environments (WALLS-ABCDF) transfer to more complex 3D reconstructions of indoor environments (MESH)?

The evaluation and comparison is conducted using a set of eight independently-trained policy networks. Two policies are trained using a **single layout** ($\Pi_{WALLS-B}$, $\Pi_{WALLS-F}$). Four policies are training using the proposed multi-layout training regime, in which the agent is randomly located in one of the layouts for each episode. The **multi-layout** polices are $\Pi_{T1}$ trained on WALLS-ABF, $\Pi_{T2}$ trained on WALLS-ABD, $\Pi_{T3}$ trained on WALLS-ADF, and $\Pi_{T4}$ trained on WALLS-ADE. Note that $\Pi_{T1}$ has been exposed to a corridor (A), door exit (B) and crosswalk (F), whereas $\Pi_{T2}$ doesn't see the crosswalk, $\Pi_{T3}$ doesn't see the door exit, and $\Pi_{T4}$ doesn't experience the crosswalk or door configurations during training. Finally, the MESH environments are used to train the policies $\Pi_{MESH-MARKET}$ and $\Pi_{MESH-HOME}$, separately using the supermarket and apartment 3D reconstructions, respectively.

For each experimental session, we train the policy for 200,000 training steps using 4 parallel simulation environments. We use the SAC implementation available on TFagents [34] and the available software integration with iGibson [2] and openAI Gym [35].

**Metrics:** The following metrics are used to evaluate the objective performance of the system. **Success rate (S)**: percentage (%) of episodes where the robot reaches its goal without colliding or timing out; **Collision rate (C)**: percentage (%) of episodes during which the robot collides with either a pedestrian or static object such as a wall; **Collision with pedestrians (PC)**: percentage of episodes involving collisions with pedestrians; **Collision with obstacles (OC)**: percentage (%) of episodes involving collisions with static obstacles such as walls; **Timeout rate (TO)**: percentage (%) of episodes where the robot runs out of time before reaching its goal; **Personal Space Overlap (PSO)**: distance (cm) between the robot and pedestrians for which the robot is closest than the specified personal space threshold, aggregated across all pedestrians and averaged over length of the episode.

### A. Experiments

*1) **Single- and Multi-Layout WALLS- Training**:* The single-layout and multi-layout policies are deployed in unseen layouts WALLS-G, WALLS-H and WALLS-I. Layout WALLS-H represents the generalization target with most compositional elements on it (corridors, doors, obstacles, intersections). The results are summarized in Tables I and II, respectively. All presented Tables contain the average results over three experimental runs of 100 episodes each. Success rate below 90% are highlighted in orange.

First, we note that policies from single-layout training, which achieve $S > 90\%$ when tested in the same training environment, have a performance drop when tested on more complex layouts. Both $\Pi_{WALLS-B}$ and $\Pi_{WALLS-F}$ had lower performance in layout WALLS-H. This indicates the need for a larger body of experience during training, considering other arrangements of constraints and the patterns of pedestrian motion that arise in these configurations. Unlike single-layout training, using multiple canonical environments for training policy $\Pi_{T1}$ resulted in consistent generalization performance to all three test environments with average success rate of 95.16% (93.5% WALLS-I, 96% WALLS-H and 96% WALLS-G) (Table II).

[1]We have previously tested that policy networks that output linear and angular velocity commands can smoothly drive our robot platform and we expect successful real robot deployment in the future. In terms of sensing, the profile of the simulated lidar accurately reflect that of the real robot's.

[2]We thank the Stanford HAI-AWS Cloud Credits for Research program for its support with compute resources.

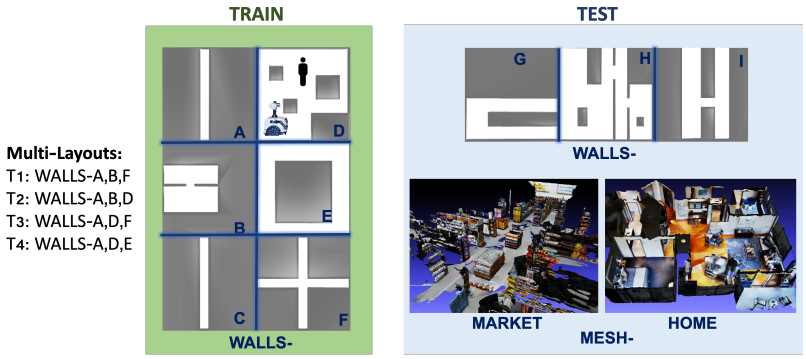



Fig. 4: Definition of the layouts used in training and testing environments in simulation.

TABLE I: Evaluation results using single-layout training

| | | Train | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | F | | | | | | B | | | | | |
| **Test** | **# ped.** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** |
| **WALLS-I** | **4** | 96.00 | 4.00 | 4.00 | 0.00 | 0.00 | 2.07 | 86.00 | 14.00 | 8.00 | 6.00 | 0.00 | 1.96 |
| **WALLS-H** | **6** | 70.00 | 30.00 | 2.00 | 28.00 | 0.00 | 0.66 | 80.00 | 20.00 | 2.00 | 18.00 | 0.00 | 1.30 |
| **WALLS-G** | **3** | 92.00 | 8.00 | 6.00 | 2.00 | 0.00 | 1.09 | 68.00 | 32.00 | 12.00 | 20.00 | 0.00 | 2.80 |

TABLE II: Evaluation results using multi-layout training

| | | $\Pi_{T1}$ | | | | | | $\Pi_{T2}$ | | | | | | $\Pi_{T3}$ | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Test** | **# ped.** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** |
| **WALLS-I** | **4** | 93.50 | 6.00 | 3.50 | 2.50 | 0.50 | 2.21 | 92.00 | 8.00 | 2.00 | 6.00 | 0.00 | 1.75 | 98.00 | 2.00 | 2.00 | 0.00 | 0.00 | 1.99 |
| **WALLS-H** | **6** | 96.00 | 4.00 | 3.33 | 0.67 | 0.00 | 1.42 | 88.00 | 12.00 | 6.00 | 6.00 | 0.00 | 1.44 | 94.00 | 6.00 | 4.00 | 2.00 | 0.00 | 1.51 |
| **WALLS-G** | **3** | 96.00 | 4.00 | 4.00 | 0.00 | 0.00 | 1.72 | 92.00 | 8.00 | 4.00 | 4.00 | 0.00 | 1.76 | 96.00 | 4.00 | 2.00 | 2.00 | 0.00 | 1.85 |

| | | $\Pi_{T4}$ | | | | | | ROS-NAV | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Test** | **# ped.** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** |
| **WALLS-I** | **4** | 74.00 | 26.00 | 10.00 | 16.00 | 0.00 | 1.98 | 91.00 | 3.00 | 2.00 | 1.00 | 6.00 | 0.00 |
| **WALLS-H** | **6** | 78.00 | 22.00 | 6.00 | 16.00 | 0.00 | 1.46 | 89.00 | 4.00 | 0.00 | 4.00 | 7.00 | 13.17 |
| **WALLS-G** | **3** | 96.00 | 4.00 | 4.00 | 0.00 | 0.00 | 1.05 | 92.00 | 3.00 | 3.00 | 0.00 | 5.00 | 0.00 |

However, not all multi-layout policies generalize equally well to the compositional test environments. While $\Pi_{T1}$ and $\Pi_{T3}$ produce consistent transfer performance (on WALLS-G, WALLS-H, and WALLS-I), $\Pi_{T2}$ and $\Pi_{T4}$, which haven't experience the crosswalk or door configurations, have significant performance drop in WALLS-H which challenges the agent the most. $\Pi_{T4}$ also has a performance cost on WALLS-I. This results supports the importance of relevant geometric configurations in enabling transfer to a compositional domain, as a key element on learning beyond simply large collection of varied experience.

The multi-layout approach results in consistent training curves as shown in Figure 5 for $\Pi_{T1}$ training with 4 random seeds. The resulting behavior of the policy can be observed in a video accompanying this paper linked here [6].

*2) **Comparison with a planning-based approach**:* Planning approaches are suitable for application in our problem set up as they can also perform in arbitrary layouts (maps) without increasing complexity as a function of the layout geometry or size in an intractable manner. We compare the performance of the learned policy $\Pi_{T1}$ with the planner available with the ROS Navigation framework (ROS-NAV-STACK) . The planner in the ROS-NAV-STACK is a layered costmap method [36] that also uses two components: a global planner using Dijkstra's algorithm and a local controller that uses the dynamic window approach [37]. The ROS navigation parameters were tuned to optimize the performance of the planner in the WALLS layouts. The tests are conducted on WALLS-I with an increasing number of pedestrians in comparison with $\Pi_{T1}$. Results are summarized on Table IV and the success rate as a function of the number of pedestrians is shown in Fig. 6. The planner resulted in lower success rate across all tests caused mainly due to timeouts. This robot-freezing problem [15] becomes more frequent as the pedestrian density increases and the planner fails to find feasible plans in the dynamic scene. While the combination of planning and learning in $\Pi_{T1}$ has a performance that also degrades with the number of pedestrians, the failure cases are due to collisions and not timeouts. Note that real deployment can count with an additional sensing-based safety stop.

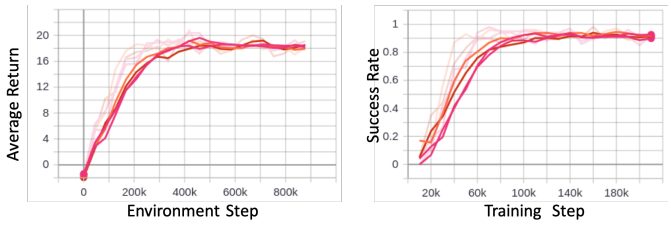



Fig. 5: $\Pi_{T1}$ training curves: Average return as a function of environment steps and the success rate for interleaved evaluations every 10,000 training steps.

The comparison with a motion planner (no learning) is a proxy to quantify the benefit added by the RL components (Fig. 6). The planner is less successful at accomplishing the goal because it suffers frequently from the robot freezing problem (large timeouts (TO) and low collisions – due to not moving – in Table IV), which means that it continuously fails to find a feasible path and keeps re-planning. In contrast, out method resulted in no timeouts.

TABLE III: Cross-domain generalization with WALLS and MESH environments

| | | $\Pi_{T1}$ | | | | | | MESH-MARKET | | | | | | MESH-HOME | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Test** | **# ped.** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** |
| **MESH-MARKET** | **6** | 77.00 | 23.00 | 15.50 | 7.50 | 0.00 | 1.96 | 75.33 | 24.67 | 21.33 | 3.33 | 0.00 | 2.19 | 78.00 | 20.00 | 20.00 | 0.00 | 0.00 | 1.54 |
| **MESH-HOME** | **6** | 66.00 | 34.00 | 10.00 | 24.00 | 0.00 | 1.13 | 43.33 | 56.67 | 18.00 | 38.67 | 0.00 | 1.66 | 84.00 | 16.00 | 14.00 | 2.00 | 0.00 | 1.93 |
| **WALLS-H** | **6** | 96.00 | 4.00 | 3.33 | 0.67 | 0.00 | 1.42 | 88.00 | 11.00 | 6.00 | 5.00 | 1.00 | 1.34 | 64.00 | 36.00 | 9.00 | 27.00 | 0.00 | 0.89 |

TABLE IV: Comparison with ROS-NAV planner

| | | $\Pi_{T1}$ | | | | | | ROS-NAV | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **Test** | **# ped.** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** | **S** | **C** | **PC** | **OC** | **TO** | **PSO** |
| **WALLS-I** | **4** | 98.00 | 2.00 | 2.00 | 0.00 | 0.00 | 1.69 | 90.00 | 2.50 | 2.00 | 0.50 | 7.00 | 0.00 |
| **WALLS-I** | **6** | 95.50 | 4.50 | 4.00 | 0.50 | 0.00 | 2.84 | 77.50 | 3.50 | 1.00 | 2.50 | 19.00 | 0.00 |
| **WALLS-I** | **8** | 88.50 | 11.50 | 10.00 | 3.50 | 0.00 | 3.99 | 74.00 | 3.00 | 1.00 | 2.00 | 23.00 | 1.35 |
| **WALLS-I** | **12** | 88.00 | 12.00 | 11.50 | 0.50 | 0.00 | 5.38 | 70.33 | 2.67 | 1.33 | 1.33 | 25.67 | 0.62 |
| **WALLS-I** | **16** | 80.50 | 19.50 | 19.00 | 0.50 | 0.00 | 6.54 | 62.00 | 2.67 | 1.00 | 1.67 | 35.00 | 0.97 |

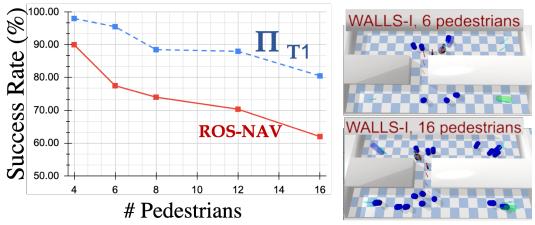



Fig. 6: Performance of T1 and ROS-NAV tested on WALLS-I with increasing number of pedestrians.

*3) **Comparison with a Reinforcement Learning Approach**:* Since our target domain in constrained environments differs from previous works in open spaces, previous RL methods would require significant modifications to include the additional needed perceptual input to be comparable in our domain (refer to RL in Section II). To establish a baseline comparison with an existent RL architecture we test our policy $\Pi_{T1}$ on the domain provided by [7], where SARL achieved state of the art results for crowd navigation (including in comparison with relevant methods such as CADRL [23]). SARL performs in open spaces and offers an open source implementation [7], including a widely-used comparative domain that consists of a circle crossing scenario in which pedestrians starts at randomized positions on a circle with $4m$ radius and navigate to the antipodal point in the circle. The SARL policy is trained and tested on this domain [7], whereas we deploy our original policy $\Pi_{T1}$ which has not seen the circle domain. Both SARL and our policy achieve 100% success rate in this domain, which establishes that our system is equally able to solve previous RL domains in the literature for the case of cooperative ORCA pedestrians. Analogous to SARL experiments in [7], Figure 7 shows some illustrative examples of $\Pi_{T1}$ navigation episodes. The robot lower speeds while pedestrians cross and then advances with increased speed. The policy allows linear velocities in the range $(-0.2, 1)m/sec$, which enable stopping and backwards motions to accommodate pedestrians.

*4) **Cross-domain generalization with WALLS and MESH environments**:* We investigate if an agent trained in the simplified representation WALLS- is capable of navigating around pedestrians in realistic 3D scanned scenes (MESH), such as a supermarket (MESH-MARKET) and an apartment (MESH-HOME), illustrated in Fig. 4 and shown in the accompanying video [6]. First we measure the performance of training and testing on MESH-MARKET and MESH-HOME (i.e. MESH to MESH), and compare to that of $\Pi_{T1}$ in the mesh environments (i.e. multi-layout training on WALLS to MESH), and viceversa. The results are presented on Table III.

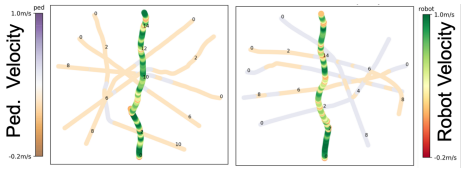



Fig. 7: Example test episodes of $\Pi_{T1}$ in the circular open domain.

The average success rate obtained by training and testing on the same mesh results in 79.5% (MESH-MARKET =75%; MESH-HOME=84%), superior to the average transfer from WALLS to MESH of 71.5% ($\Pi_{T1}$ in MESH-MARKET =77%; $\Pi_{T1}$ in MESH-HOME=66%). This is consistent with the expected performance of training in the target environment vs. policy transfer. However, transfer from MESH training to any other environments exhibits a significant performance drop compared to $\Pi_{T1}$. This indicates that the MESH performance is the product of overfitting to the environment, while WALLS training performs more consistently across domains. This result supports both the layout compositional argument and the approach of training on abstracted walls worlds .

## V. Conclusion

We present an approach based on reinforcement learning to learn robot policies for navigation around pedestrians in constrained environments. The proposed model uses a motion planner that provides a globally planned trajectory, whereas the reinforcement component handles the local interactions needed for on-line adaptation to pedestrians. The learned policy exhibits interesting behaviors such as slowing down, maneuvering around pedestrians and going backwards to accommodate different common interactions that emerge as a consequence of the constrained layouts.

The analysis of the use of a compositionality principle for the proposed multi-layout training regime showed (1) the ability to train on simplified model abstractions and deploy on a more complex unseen environment with composition of the basic layouts; (2) that policies trained on relevant geometric configurations enable better generalization than those trained on layouts that don't exhibit the geometry and inherent pedestrians interactions present in the target layout; and (3) policies trained on walls-worlds are able to generalize to unseen 3D reconstructions from real environments (such as an apartment and a supermarket). These results offer a practical advantage by removing the need to train on target environments with high reconstruction fidelity.

## REFERENCES


[1] F. Xia, A. R. Zamir, Z.-Y. He, A. Sax, J. Malik, and S. Savarese, "Gibson env: real-world perception for embodied agents," in *Computer Vision and Pattern Recognition (CVPR), 2018 IEEE Conference on*, IEEE, 2018.

[2] F. Xia, C. E. Li, W. Shen, P. Kasimbeg, R. Martin-Martin, A. Toshev, and S. Savarese, "Interactive gibson: Benchmark for interactive navigation in cluttered environments," *RA-L*, 2020.

[3] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning," 2016.

[4] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics research*, pp. 3–19, Springer, 2011.

[5] D. Helbing and P. Molnar, "Social force model for pedestrian dynamics," *Physical review E*, vol. 51, no. 5, p. 4282, 1995.

[6] *Project Website and accompanying video*. 2020. `https://ai.stanford.edu/~cdarpino/socialnavconstrained/`.

[7] C. Chen, Y. Liu, S. Kreiss, and A. Alahi, "Crowd-robot interaction: Crowd-aware robot navigation with attention-based deep reinforcement learning," in *2019 International Conference on Robotics and Automation (ICRA)*, pp. 6015–6022, IEEE, 2019.

[8] Y. Che, A. M. Okamura, and D. Sadigh, "Efficient and trustworthy social navigation via explicit and implicit robot–human communication," *IEEE Transactions on Robotics*, vol. 36, no. 3, pp. 692–707, 2020.

[9] H. Khambhaita and R. Alami, "Viewing robot navigation in human environment as a cooperative activity," in *Robotics Research*, pp. 285–300, 2020.

[10] V. V. Unhelkar, C. Pérez-D'Arpino, L. Stirling, and J. A. Shah, "Human-robot co-navigation using anticipatory indicators of human walking motion," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6183–6190, IEEE, 2015.

[11] Y. Chen, C. Liu, B. E. Shi, and M. Liu, "Robot navigation in crowds by graph convolutional networks with attention learned from human gaze," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2754–2761, 2020.

[12] C. Mavrogiannis, A. M. Hutchinson, J. Macdonald, P. Alves-Oliveira, and R. A. Knepper, "Effects of distinct robot navigation strategies on human behavior in a crowded environment," in *2019 14th ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pp. 421–430, IEEE, 2019.

[13] S. M. LaValle, *Planning algorithms*. Cambridge university press, 2006.

[14] V. Kunchev, L. Jain, V. Ivancevic, and A. Finn, "Path planning and obstacle avoidance for autonomous mobile robots: A review," in *International Conference on Knowledge-Based and Intelligent Information and Engineering Systems*, pp. 537–544, Springer, 2006.

[15] P. Trautman and A. Krause, "Unfreezing the robot: Navigation in dense, interacting crowds," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 797–803, IEEE, 2010.

[16] P. Trautman, J. Ma, R. M. Murray, and A. Krause, "Robot navigation in dense human crowds: Statistical models and experimental studies of human–robot cooperation," *The International Journal of Robotics Research*, vol. 34, no. 3, pp. 335–356, 2015.

[17] M. W. Otte, "A survey of machine learning approaches to robotic path-planning," *Cs. Colorado. Edu*, 2015.

[18] D. A. Pomerleau, "Alvinn: An autonomous land vehicle in a neural network," in *Advances in neural information processing systems*, pp. 305–313, 1989.

[19] U. Muller, J. Ben, E. Cosatto, B. Flepp, and Y. L. Cun, "Off-road obstacle avoidance through end-to-end learning," in *Advances in neural information processing systems*, pp. 739–746, 2006.

[20] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, *et al.*, "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.

[21] A. Pokle, R. Martín-Martín, P. Goebel, V. Chow, H. M. Ewald, J. Yang, Z. Wang, A. Sadeghian, D. Sadigh, S. Savarese, *et al.*, "Deep local trajectory replanning and control for robot navigation," in *2019 International Conference on Robotics and Automation (ICRA)*, pp. 5815–5822, IEEE, 2019.

[22] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.

[23] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," in *2017 IEEE international conference on robotics and automation (ICRA)*, pp. 285–292, IEEE, 2017.

[24] Y. F. Chen, M. Everett, M. Liu, and J. P. How, "Socially aware motion planning with deep reinforcement learning," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 1343–1350, IEEE, 2017.

[25] R. A. Epstein, E. Z. Patai, J. B. Julian, and H. J. Spiers, "The cognitive map in humans: spatial navigation and beyond," *Nature neuroscience*, vol. 20, no. 11, p. 1504, 2017.

[26] W. Gao, D. Hsu, W. S. Lee, S. Shen, and K. Subramanian, "Intention-net: Integrating planning and deep learning for goal-directed autonomous navigation," in *Proceedings of the 1st Annual Conference on Robot Learning*, vol. 78 of *Proceedings of Machine Learning Research*, pp. 185–194, 2017.

[27] J. Choi, C. Dance, J. eun Kim, K. sik Park, J. Han, J. Seo, and M. Kim, "Fast adaptation of deep reinforcement learning-based navigation," in *2020 International Conference on Robotics and Automation (ICRA)*, IEEE, 2020.

[28] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *ICML '09*, 2009.

[29] T. Matiisen, A. Oliver, T. Cohen, and J. Schulman, "Teacher-student curriculum learning," *IEEE transactions on neural networks and learning systems*, 2019.

[30] A. Graves, M. G. Bellemare, J. Menick, R. Munos, and K. Kavukcuoglu, "Automated curriculum learning for neural networks," in *Proceedings of the 34th International Conference on Machine Learning-Volume 70*, pp. 1311–1320, 2017.

[31] C. Florensa, D. Held, M. Wulfmeier, M. Zhang, and P. Abbeel, "Reverse curriculum generation for reinforcement learning," in *Conference on Robot Learning*, pp. 482–495, 2017.

[32] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning*, vol. 80, pp. 1861–1870, 2018.

[33] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, *et al.*, "Soft actor-critic algorithms and applications," *arXiv preprint arXiv:1812.05905*, 2018.

[34] S. Guadarrama, A. Korattikara, O. Ramirez, P. Castro, E. Holly, S. Fishman, K. Wang, E. Gonina, N. Wu, C. Harris, *et al.*, "Tf-agents: A library for reinforcement learning in tensorflow," 2018.

[35] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," *arXiv preprint arXiv:1606.01540*, 2016.

[36] D. V. Lu, D. Hershberger, and W. D. Smart, "Layered costmaps for context-sensitive navigation," in *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 709–715, IEEE, 2014.

[37] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.

## SUPPLEMENTARY MATERIALS

### I. SIMULATION ENVIRONMENT IMPLEMENTATION

*a) **Simulation**:* The robot collects experience in the simulated environment while being tasked to navigate from a set of initial to target positions. A policy network is learned using the off-policy Soft Actor Critic reinforcement algorithm (SAC) in continuous action space of robot velocity. The policy network receives guidance from a motion planner that provides waypoints to follow a globally planned trajectory, whereas the reinforcement component handles the balance with the local interactions. The RL agent is rewarded with a balance between following the waypoints but also deviating to accommodate for the presence of pedestrians. This framework results in policies that can adapt to arbitrary environments by generating a navigation motion plan, and leaves the local reactive interaction with nearby pedestrians to be learned by reinforcement. Robot perception is based on a lidar sensor, which raw data resulted in enough information for obstacles and pedestrians avoidance in the tested environments.

The simulation environment is built based on iGibson and the PyBullet physics engine. The learning agent is a differential drive mobile robot. These non-holonomic constraints are included in the simulation. The policy network outputs the desired values of linear and angular velocity for the robot $(V, w)$, which are passed to a low level velocity controller in wheel speed. This software architecture in simulation follows the architecture on the real robot in terms of input commands.

*b) **Robot Behavior**:* The policy network receives as input a set of observations $o = \{goal, lidar, waypoints\}$, where *goal* is the episodic navigation goal (2D polar coordinates of the goal in robot frame), *lidar* contains 1D LiDAR sensor measurements in robot frame, and *waypoints* contains a number of *reference waypoints* computed by a global planner with access to a map of the environment. The waypoints are selected from the collision-free shortest path between the start and goal points of the episode. The *reference waypoints* are updated as the robot makes progress along the desired path. A *waypoint resolution* is used to define the distance between consecutive waypoints, and a *waypoint tolerance* value is used to define the minimum proximity from the robot to a waypoint to receive the waypoint reward and update the next desired waypoint. Parameters for the robot behavior are summarized on Table V.

TABLE V: Simulator, agent and pedestrians

| | |
|---|---|
| **Observations and Robot Behavior** | |
| Angular Velocity | $[-0.5, 0.5] rad/sec$ |
| Linear Velocity | $[-0.2, 1] m/sec$ |
| Number of Reference Waypoints | 6 |
| Waypoint Resolution | 1.00m |
| Waypoint Tolerance | 0.50m |
| **Pedestrians Behavior (ORCA)** | |
| Personal Space | $10cm$ |
| Pedestrian Radius | $30cm$ |
| Prefered Velocity | $2m/sec$ |
| Maximum Speed | $2m/sec$ |
| Robot Visible to Pedestrians | TRUE |

*c) **Simulated Pedestrians**:* The simulated pedestrians follow the ORCA social forces model. The used parameters are summarized on Table V. The ORCA model jointly optimizes the trajectories of all agents (pedestrians) to collaboratively avoid collisions. This property is guaranteed if all agents on the scene participate in the optimization. The ORCA model that drives all pedestrians has access to the state of the robot and considers it a participant agent. This produces the behaviors of the pedestrians to be collaborative with the robot under the expectation that the robot will be reciprocally collaborative in avoiding collisions. However, the robot follows a policy different from ORCA (either the learned policy or the ROS-NAV-STACK planner). This simulation strategy results in an approximation to mutual collaboration between the pedestrians and the robot. While ORCA offers an efficient model to simulate multiple pedestrians, it is limited in the types of trajectories the pedestrians can follow and does not adapt well to trajectory that require turns, concave obstacles or trajectories with lack of visibility to the goal. To address this limitation, each layout includes a definition of allowed initial and end areas (2D locations and allowed radius shown in Fig. 8 as shaded areas) to require underlying pedestrian trajectories that are feasible for the ORCA model to generate. For each episode, locations are randomly sampled from this allowed set. Pedestrians are represented in the simulator interchangeable as either a cylinder shape or a rigid mesh. We observe that improvements on the behavioral simulation of pedestrians at the macro level would potentially unlock greater adaptability of the learned behaviors to the real world. For example, in the simulation of MESH- worlds, which follows the same principles as the WALLS- environments, the simulation of pedestrians becomes more challenging as the specific context becomes relevant for sampling the start and end points for the ORCA-driven pedestrians: (e.g. walking from the kitchen to the bedroom). We expect that improvement on pedestrian simulation alone will result on better robot performance overall, without additional modifications to the method, as it would produce more fluent pedestrians trajectories. We leave these improvements as subject of future work.

*d) **Multi-Layout Training**:* We train a set of policies on selected combinations of simple layouts WALLS-. These combinations are denominated $\{T1, T2, T3, T4\}$. For each $T_i$, the robot is randomly located in one of the included WALLS- layouts at the beginning of each training episode.

*e) **Simulation Design**:* Using the proposed approach and generally on simulation-based robot learning, the engineering effort centers on designing a simulation environment that properly exhibits the dynamics and properties of the target domain. In robotics, this effort differs from planning techniques, in which the effort is rather centered in explicitly defining algorithmic solutions within some problem properties. Both approaches require significant design and engineering, while the difference lies in specifying the environment (the problem) or the algorithm (an explicit solution).

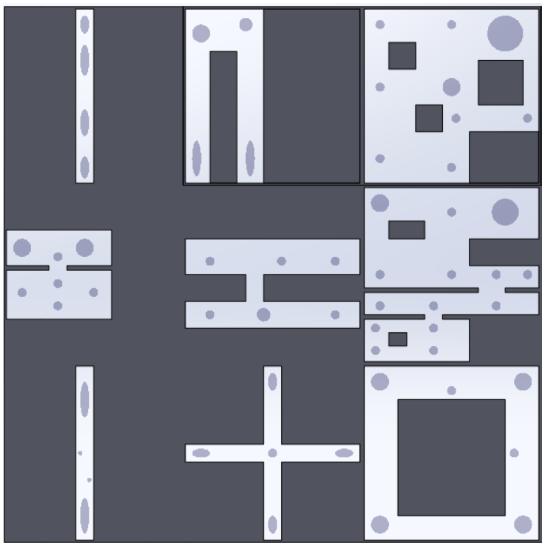


Fig. 8: Locations on WALLS- domains. Each layout has dimensions $20mx20m$.

## II. HYPERPARAMETERS

Table VI reports the hyper-parameters used for the reinforcement learning algorithm SAC. All the reported policies were learned using the same set of hyper-parameters.

We tuned the parameters of the ROS-NAV-STACK planner to optimize its performance in the tested environments. Adjusting these values reduces planner timeouts and increases likelihood of finding feasible solutions in the footprint of the tested layouts. Table VII reports the used parameters.

TABLE VI: SAC Hyperparameters

| **SAC Hyperparameters** | |
|---|---|
| Hardware Configuration | NVIDIA® V100 |
| Optimizer | Adam |
| Discount Ratio Gamma | 0.99 |
| Critic Learning Rate | 0.0003 |
| Actor Learning Rate | 0.0003 |
| Alpha Learning Rate | 0.0003 |
| Maximum Episode Lenght (Step) | 500 |
| Replay Buffer Capacity | 150000 |
| Action Time Step | $0.25sec$ |

TABLE VII: Optimized parameter for ROS-NAV planner

| **DWA Trajectory Scoring Parameters** | |
|---|---|
| path_distance_bias | 48 |
| goal_distance_bias | 24.00 |
| occdist_scale | 0.01 |
| **Global and local costmaps** | |
| footprint_padding | 0.05 |
| inflation_layer: cost_scaling_factor | 2.58 |
| inflation_radius | 2.50 |
| **move_base Parameters** | |
| planner_frequency | 0.5 Hz |
| controller_frequency | 15 Hz |
| planner_patience | 200 sec |
| controller_patience | 15 sec |
| recovery_behavior_enabled | FALSE |
| clearing_rotation_allowed | FALSE |
| max_planning_retries | -1 |